Name :- Aaryan N. Rodrane Roll no :- 5004
branch :- AI-DS batch : A1 SAPID :- 60018250070

Experiment No 1

Title :- Data-Types, operators and input-output in Python

Aim :- To implement a program to demonstrate data types operator and input output in Python

Theory :-

* Basic Data types in Python with Examples:

Python is dynamically typed, interpreted language. Unlike statically typed languages such as C or Java, Python determines the data type of a variable at runtime based on the value assigned to it. Every value in Python is an object, and each object belongs to a specific class.

Python provides the following built-in data types:

1. Int (Integer):

The int type stores whole numbers without a decimal component. Python integers have arbitary precision, meaning they are not constrained by a fixed bit width like 32-bit or 64-bit integers in other languages.

Sundaram FOR EDUCATIONAL USE

code:
```
x = 10
y = -25
z = 0
print (type(x)) # <class 'int'>
print (x + y) # [output: -15]
```

2. float (floating-point Number)

The float type represents real numbers with a decimal point. Internally, Python floats follow the IEEE 754 double-precision standard, which provides approximately 15 to 17 significant decimal digits of precision.

code:
```
pi = 3.14159
temperature = -7.5
print (type(pi)) # output: <class 'float'>
print (pi * 2) # output: 6.28318
```

3. complex

The complex type stores numbers of the form $a + bj$, where a is the real part and b is the imaginary part. Python uses j to denote the imaginary unit.

code:
```
c = 3 + 4j
print (c.real) # output: 3.0
print (c.imag) # output: 4.0
print (type(c)) # output: <class 'complex'>
```

Sundaram
FOR EDUCATIONAL USE

4. str (string)

A string is an ordered, immutable sequence of Unicode characters. Strings can be enclosed in single Quotes, double Quotes or triple Quotes (for multiline strings).

code :
```
name = "Python"
message = 'Hello, World!'
multiline = """This is a
multiline string."""
print (type(name)) # output : <class 'str'>
print (name[0]) # output : P
print (len(name)) # output : 6
```

5. Bool (Boolean)

The Bool type holds one of two values : True or False. Booleans are technically a subclass of int in Python, where True evaluates to 1 and False evaluates to 0

code :
```
is_active = True
is_empty = False
print (type(is_active)) # output : <class 'bool'>
print (is_active + 6) # output : 7
print (int(is_empty)) # output : 0
```

Sundaram
FOR EDUCATIONAL USE

*) Basic Operators in Python with Examples

Operators are tokens that instruct the interpreter to perform a specific computation on one or more operands. Python supports several categories of operators.

1. Arithmetic Operator

These operators perform standard mathematical computations. The floor division operator (//) returns the quotient rounded down to the nearest integer, and the modulus operator (%) returns the remainder.

code:-

```
a = 10
b = 3

print(a+b) # Addition output: 13
print(a-b) # Substraction output: 7
print(a*b) # Multiplication output: 30
print(a/b) # Division output: 3.3333333....
print(a//b) # floor Division output: 3
print(a%b) # Modulus Output: 1
print(a**b) # Exponent output: 1000
```

Sundaram
FOR EDUCATIONAL USE

2. Comparison (Relational) operators

Comparison operators evaluate the relationship between two operands and return a Boolean value (True or False). These are widely used in conditional statements and loop control.

Code:-

```
x = 5
y = 10

print(x == y) # equal to output : False
print(x != y) # Not equal to output : True
print(x > y) # greater than output : False
print(x < y) # Less than output : True
print(x >= y) # greater or equal to output : False
print(x <= y) # Less or equal to output : True
```

3. Assignment operators

Assignment operators binds value to a variable. Compound assignment operators (such as += and *=) combine an arithmetic operation with assignment into a single statement.

Code:-

```
a = 10
a += 5 # Equivalent to a = a+5 → a = 15
a -= 3 # Equivalent to a = a-3 → a = 12
a *= 2 # Equivalent to a = a*2 → a = 24
a //= 4 # Equivalent to a = a//4 → a = 6
print(a) # output : 6
```

Sundaram
FOR EDUCATIONAL USE

4. Logical Operators

Logical operators combine or negate Boolean expressions. Python evaluates and and or using short-circuit evaluation: it stops evaluating as soon as the result is determined.

code:
```
p = True
q = False

print(p and q) # both must be True output: False
print(p or q) # at least one true output: True
print(not p) # Negation output: False
print(not q) # Negation output: True
```

5. Bitwise Operators

Bitwise operators operate directly on the Binary representation of integer operands. They are used in low-level programming, cryptography and performance-sensitive applications.

code:
```
a = 5    # binary : 0101
b = 3    # binary : 0011

print(a & b) # AND output: 1 (0001)
print(a | b) # OR output: 7 (0111)
print(a ^ b) # XOR output = 6 (0110)
print(~a) # NOT output: -10 (1010)
print(a << 1) # left shift output: 10 (1010)
print(a >> 1) # Right shift output: 2 (0010)
```

Sundaram
FOR EDUCATIONAL USE

5. Membership Operators

Membership operators test whether a value is present in a sequence such as a list, tuple, string or set. They return a Boolean result.

code:- fruits=["apple", "Banana", "mango"]

print ("apple" in fruits) # output: True
print ("grape" in fruits) # output: False
print ("grape" not in fruits) # output: True

text = "Python Programming"
print ("Python" in text) # output: True

6. Identity Operators

Identity operators check whether two variables refer to the same object in memory. This is different from equality: two objects can hold the same value but occupy different memory locations.

code: a = [1, 2, 3]
b = a # b refers to the same object as a
c = [1, 2, 3] # c is a new object with the same value
print (a is b) # output: True (same object in memory)
print (a is c) # output: False (different objects)
print (a == c) # output: True (same values)
print (a is not c) # output: True

Sundaram
FOR EDUCATIONAL USE

3. Input and Output in Python

Python provides built in functions for reading data from the user and displaying results to the console.

* Output using print()

The print() function writes data to the standard output stream. It accepts multiple positional arguments and two keywords: sep (separator between arguments, default is a space) and end (character printed after the last argument, default is a newline)

Syntax: print (object(s), sep=' ', end='\n')

code:
```
print ("Hello, World!") # output: Hello, World!
print (10, 20, 30)  # output: 10 20 30
print (10, 20, 30, sep=":", ) #output: 10, 20, 30
print ("First", end = " ")
print ("second") # output: First second
print ("Name:", "Carlson", "Age:", 20)
    # output: Name: Carlson Age: 20
```

* formatted output

Python supports three methods for formatted string output:

Sundaram
FOR EDUCATIONAL USE

Method 1 :- % formatting (old style)

code :-
```
name = "Aaryan"
marks = 95.5
print ("student : %s, Marks : %.2f" %
       (name, marks))
#output : student : Aaryan , Marks : 95.50
```

Method 2 :- str.format() method

code :-
```
print ("Student : {} , Marks : {:.2f}".format(name,
       marks))
#output : student : Aaryan, Marks : 95.50
```

Method 3 :- f-strings (Python 3.6 and above)

code :-
```
print (f "Student : {name}, Marks : {marks:.2f}")
#output : Student : Aaryan , Marks : 95.50
```

f-strings are the most readable and efficient of the three methods. they allow expressions to be embedded directly inside the string literal

* Input using input()

The input() function pauses program execution and reads a line of text from the standard input stream. It always return the value as a string (str), regardless of what the user types.

Sundaram
FOR EDUCATIONAL USE

Syntax: input ([prompt])

code:
```
name = input("Enter your name: ")
print("Hello, ", name)
# input : Karl
# output : Hello, Aaryan.
```

* Type Conversion (casting)

Since input() returns a string, numeric input must be explicitly converted using type-casting functions.

code:
```
age = int(input("Enter your age: "))
cgpa = float(input("Enter your CGPA: "))
print(f"Age: {age}, CGPA: {cgpa:.2f}")
```

Python's built-in type conversion functions:

1. int(x) : converts x to an integer
2. float(x): converts x to a floating-point number
3. str(x) : converts x to a string.
4. bool(x): converts x to a Boolean
5. complex(x): converts x to a complex number.

code:
```
x = "42"
print(int(x))  # output: 42
print(float(x)) # output: 42.0
print(bool(0)) # output: False
print(bool(99)) # output: True
print(str(3.14)) # output: '3.14'
```

Sundaram
FOR EDUCATIONAL USE

* Python input - output code :-

```
length = float(input("Enter length : "))
breadth = float(input("Enter breadth : "))
area = length * breadth
perimeter = 2 * (length + breadth)

print(f"\n Rectangle Details")
print(f"length : {length}")
print(f" breadth : {breadth}")
print(f"Area : {area :.2f} sq. units")
print(f"Perimeter : {perimeter :.2f} units")
```

output: Enter length : 5.0
Enter breadth : 3.0

Rectangle Details
length : 5.0
breadth : 3.0
Area : 15.00 sq. units
Perimeter : 16.00 units

* Conclusion :-

Through this experiment, the Basic data types in Python such as int, float, complex, str and bool were studied along with their usage in programs. The various operators like arithmetic, logical, relational and bitwise were understood and implemented with examples. The input() and print() functions were used to handle user input and display formatted output on the console.

Sundaram
FOR EDUCATIONAL USE